ALTRUS – Adaptive Life-support and Therapeutic Robotic Unit System

Praveen S.K.S
Faculty of Computing
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22114044@my.sliit.lk

S.N.S.B.K.K. Semasinghe
Faculty of Computing
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22084668@my.sliit.lk

Madushanka L.A.S
Faculty of Computing
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22602114@my.sliit.lk

Hansi De Silva
Faculty of Computing
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
hansi.d@sliit.lk

Ashvinda Iddamalgoda
Faculty of Computing
Department of Computer Systems Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
ashvinda.i@sliit.lk

***Abstract*—Assistive mobile robots are becoming increasingly important for supporting independent living and continuous healthcare monitoring in domestic environments. However, existing approaches remain largely fragmented, with navigation, human–robot interaction (HRI), sensor integration, and system reliability often engineered as isolated subsystems, thereby increasing overall complexity, limiting reproducibility, and reducing operational robustness. This paper presents ALTRUS, a unified modular assistive robotic architecture implemented on Robot Operating System 2 (ROS 2), built upon four reusable, developer-oriented frameworks designed to enable rapid robot construction across diverse hardware configurations: (i) an adaptive navigation framework combining hybrid simultaneous localization and mapping (SLAM)-based localization with multimodal voice and gesture command fusion; (ii) a configurable firmware generation framework enabling IoT-based health monitoring through template-driven ESP32 code synthesis; (iii) a multimodal emotion-aware interaction framework incorporating facial and vocal affect recognition with context-aware empathetic response generation; and (iv) a fault-tolerant middleware backbone supporting deterministic intent arbitration, proactive subsystem recovery, and blockchain-backed immutable runtime logging. Together, these frameworks establish a layered architecture that bridges perception, cognition, interaction, and safety management within a single deployable platform. A declarative configuration model further enables rapid customization across diverse robot morphologies while preserving modularity and reusability. Experimental validation demonstrates reliable navigation, real-time multimodal interaction, continuous health data streaming, and resilient operation under induced fault conditions, providing a scalable foundation for next-generation assistive robotic development.**

***Keywords*—Assistive robotics, ROS 2, modular architecture, SLAM, IoT health monitoring, emotion-aware interaction, fault-tolerant middleware, human–robot interaction.**

I. Introduction

Recent advancements in robotics have accelerated the development of assistive systems intended to support humans in domestic, healthcare, and rehabilitation environments. These systems increasingly depend on intelligent software capable of perceiving surroundings, interpreting user intent, responding empathetically, and coordinating distributed sensing and control modules. Despite significant progress in individual algorithmic domains - such as navigation, perception, and human–robot interaction - the integration of these capabilities into reusable and scalable software architectures remains an open challenge. Many existing robotic solutions are implemented as tightly coupled, task-specific systems that are difficult to adapt, extend, or reuse across platforms and application scenarios. This fragmentation limits reproducibility, slows development cycles, and increases engineering overhead when constructing complex assistive robotic platforms.

A major limitation arises from the absence of unified software frameworks that allow independent functional components to interoperate within a cohesive system architecture. Developers frequently design navigation stacks, sensor-processing modules, emotional interaction engines, and middleware infrastructures independently, resulting in duplicated effort and incompatibility across subsystems. While modular robotics has been widely explored at the hardware level, equivalent modularity at the software framework level remains insufficiently addressed. Existing approaches often prioritize performance optimization for isolated tasks rather than architectural extensibility, producing systems that are difficult to configure, scale, or repurpose. Consequently, there is a

clear need for developer-friendly frameworks that support modular deployment, configuration-driven customization, and cross-domain integration without extensive re-engineering. To address this gap, this work presents the design and implementation of a unified collection of reusable software frameworks, collectively deployed within an assistive robotic platform referred to as ALTRUS. Rather than constructing a monolithic software stack, the proposed approach introduces four independent yet interoperable frameworks, each targeting a distinct challenge in assistive robotics while maintaining portability and configurability across robotic platforms. These frameworks were designed according to the principles of modular abstraction, interface standardization, and middleware-driven interoperability, enabling each component to function either as a standalone developer resource or as part of an integrated ecosystem.

The first framework targets adaptive robot mobility and provides a reusable navigation architecture that combines hybrid localization, dynamic obstacle awareness, and multimodal command interpretation through voice and gesture inputs. This architecture enables natural human–robot interaction while preserving robust path planning and environmental responsiveness. Its configuration-driven structure allows adaptation to different robot geometries and sensing configurations, thereby emphasizing developer accessibility and reusability beyond a single deployment. The second framework addresses heterogeneous sensor integration in assistive robotic ecosystems by introducing a community-extensible module layer. Rather than hardcoding device logic, the system standardizes sensor metadata, component packaging, and build integration through a registry-driven workflow. Each sensor is delivered as a self-contained ESP-IDF component with a defined interface and is installed via a registry manifest, allowing new sensing technologies to be incorporated without modifying the core robotic software. This abstraction provides consistent data pipelines, adapter boundaries, and scalable IoT-ready interfaces, while keeping the core architecture stable as the sensor ecosystem grows.

The third framework introduces an emotional perception and response generation module that enhances human–robot interaction through empathetic behavioral adaptation. Emphasis is placed on reusability by separating the perception, inference, and behavioral response layers. This layered structure enables developers to substitute detection models or response strategies without affecting system stability, thereby supporting flexible experimentation and deployment across diverse interaction contexts.

The fourth framework contributes a fault-tolerant middleware architecture designed to ensure dependable operation in modular robotic ecosystems. Assistive robots rely on distributed components that must maintain synchronization, resilience, and trustworthiness during runtime. The proposed middleware integrates intent-driven communication and decentralized validation mechanisms to ensure reliability, data integrity, and recoverability under partial system failures. Its reusable service orchestration model supports deployment across heterogeneous robotic configurations and network topologies.

These four frameworks were evaluated both independently and through integration within the ALTRUS robotic platform to validate interoperability and practical deployment feasibility. The ALTRUS system serves as a proof-of-concept embodiment, demonstrating how reusable frameworks can be combined to construct a functional assistive robot. Simulation-based experimentation and physical implementation verified that modular framework boundaries remain intact while enabling coordinated system behavior. This validation demonstrates the effectiveness of reusable architecture principles in reducing integration complexity while preserving performance and extensibility.

The primary contribution of this work lies not solely in algorithmic innovation, but in demonstrating a comprehensive framework-oriented approach to assistive robotics software design. By prioritizing modularity, abstraction, and interoperability, this research advances robotic software engineering toward scalable and reusable ecosystem development. Deployment within ALTRUS further illustrates how such frameworks transition from conceptual design to operational implementation, bridging the gap between academic experimentation and practical robotic application. The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the architectural principles underlying each framework and details their integration into the ALTRUS system. Section IV discusses evaluation results obtained through simulation and real-world deployment. Section V concludes the paper and outlines directions for future work.

II. RELATED WORK

The ALTRUS assistive robot serves as a deployment platform for four reusable, developer-oriented software frameworks addressing key domains of assistive robotics. These frameworks emphasize modular integration, configuration-driven customization, and cross-platform reuse across diverse robotic morphologies. Prior research has produced substantial advancements in navigation, human–robot interaction (HRI), IoT sensing, emotion-aware systems, and middleware design. However, these advancements are often developed in isolation, resulting in fragmented software ecosystems that complicate deployment and adaptation in real-world assistive contexts. This section reviews representative work across these domains and highlights the limitations that motivate the integrated, reusable software solutions implemented on ALTRUS.

A. Adaptive Navigation with Voice and Gesture Commands

Autonomous indoor navigation remains a foundational capability for assistive robots. Traditional approaches rely on simultaneous localization and mapping (SLAM) techniques for global environment understanding and path planning. Static SLAM-based systems, including TurtleBot3 navigation [1] and PLD-SLAM [2], achieve reliable mapping accuracy but typically require remapping when environmental layouts change, limiting long-term adaptability. Modern navigation

frameworks such as ROS 2 Nav2 [3] incorporate sensor-driven obstacle detection and real-time path replanning, improving performance in dynamic environments. Hybrid navigation strategies combining global localization with perception-driven motion planning have further enhanced system responsiveness [4].

Parallel research has explored natural interaction interfaces for navigation control. Speech recognition systems, including Vosk and DeepSpeech [5], enable offline command interpretation, while gesture recognition approaches leveraging MediaPipe and OpenCV [6] provide visual interaction channels. Multimodal fusion techniques have demonstrated improved robustness by combining these modalities [7], [8]. Despite these advances, most implementations treat interaction modules as external command interfaces rather than integrating human intent directly into navigation decision-making.

The adaptive navigation framework implemented on the ALTRUS robot addresses this limitation by tightly coupling hybrid SLAM-based localization with multimodal intent recognition. Voice and gesture inputs dynamically modify navigation goals during execution, enabling context-aware behavioral adjustment. Additionally, a declarative configuration pipeline automates the generation of URDF models, transformation trees, and navigation parameters across hardware variations. This reusable software approach reduces developer effort when adapting navigation capabilities to new robotic platforms and simulation environments.

B. Sensor Integration for IoT Health Devices

Assistive robotic systems increasingly incorporate IoT-based sensing for monitoring physiological and environmental conditions. Developing firmware for such devices typically involves manual implementation of sensor interfaces, sampling routines, communication protocols, and integration with analytics pipelines. Model-driven engineering solutions such as ThingML [9] and CAPSml [10] facilitate automated code generation across heterogeneous platforms; however, they are rarely integrated directly into robotic system workflows.

Health monitoring research, particularly in fall detection and activity tracking, has explored threshold-based signal analysis [11] and machine learning-based classification methods [12]. While these studies demonstrate sensing effectiveness, they often evaluate algorithms independently from robotic middleware integration, thereby limiting their applicability in holistic assistive environments.

The sensor integration framework deployed on the ALTRUS robot introduces a template-driven firmware generation mechanism that produces ESP32 code based on declarative configurations. Developers specify sensors, sampling rates, and communication settings, after which firmware is generated with built-in streaming support for Wi-Fi and Bluetooth Low Energy (BLE). Safe regeneration preserves manual modifications during iterative development. By focusing on reusable, developer-oriented software generation, this framework reduces repetitive engineering effort while enabling seamless integration of health monitoring into robotic operation.

C. Emotion Detection and Empathetic Behavior Generation

Emotion recognition technologies have advanced significantly through deep learning methods applied to visual and auditory data. Facial emotion recognition commonly employs convolutional neural networks (CNNs) trained on datasets such as FER2013 or RAF-DB [13], while speech emotion recognition relies on mel-frequency cepstral coefficient (MFCC) feature extraction combined with neural sequence models [14]. Multimodal fusion architectures, including attention-based transformers, improve cross-modal inference accuracy [15]–[17]. Concurrently, large language model (LLM)-based conversational systems have enhanced dialogue generation capabilities [18].

However, these approaches often operate independently from robotic control frameworks and lack mechanisms for maintaining contextual emotional memory across interactions. Consequently, robotic responses may fail to reflect continuity of user emotional states, reducing perceived empathy and social intelligence.

The emotion interaction framework implemented on ALTRUS performs real-time multimodal perception of facial expressions and vocal prosody while maintaining contextual emotional histories across multi-turn interactions. Locally deployed language models generate adaptive responses informed by both immediate perception and historical context, creating a closed loop between sensing and behavioral output. Its modular and reusable software structure further enables extension with alternative sensing modalities or conversational strategies.

D. Fault-Tolerant, Intent-Driven Middleware

Robotic middleware platforms provide essential communication infrastructure and distributed execution support. Frameworks such as ROS [19] and ROS 2 [20] enable modular component integration but typically do not include system-level supervision or deterministic arbitration mechanisms. Existing fault-tolerance solutions frequently rely on hardware redundancy or localized recovery strategies, and seldom address coordination among heterogeneous subsystems. Blockchain-based logging approaches have been explored for IoT integrity assurance [21], [22]; however, their application to runtime robotic coordination remains limited. Similarly, intent arbitration research improves command interpretation but seldom integrates recovery or auditing capabilities.

The middleware framework implemented within the ALTRUS robotic platform introduces deterministic arbitration of competing subsystem intents, heartbeat-based monitoring for fault detection, and recovery strategies supporting graceful degradation. Blockchain-backed logging provides immutable event tracking to enhance auditability and facilitate debugging. Designed as reusable supervisory software, the framework facilitates coordinated operation of navigation, sensing, and interaction modules while reducing overall system integration complexity.

E. Research Gap

Despite notable progress across individual domains, existing solutions remain fragmented and rarely provide reusable, developer-oriented integration across assistive robotic subsystems. This work addresses the identified gap by implementing four modular software frameworks on the ALTRUS assistive robot, forming a cohesive architecture that supports integrated deployment across navigation, sensing, interaction, and system supervision. By emphasizing configurability, extensibility, and software reuse, the proposed approach reduces development overhead while enabling adaptation across diverse hardware configurations and operational scenarios. This unified design establishes a scalable foundation for future assistive robotic system development.

III. METHODOLOGY

A. Architecture Overview

The proposed ALTRUS assistive robot system is designed as a developer-friendly and reusable robotic framework that simplifies the creation, deployment, and validation of assistive robots in both simulation and real-world environments. Rather than requiring developers to manually assemble fragmented middleware components and robot-specific software stacks, ALTRUS enables rapid robot instantiation through a configuration-driven approach.

The architecture follows a perception–decision–execution paradigm, ensuring a clear separation between sensing, reasoning, arbitration, and actuation. This separation prevents tight coupling between subsystems and allows independent evolution of navigation, embedded sensing, emotion-aware interaction, and supervisory control components without compromising global system stability.

Rather than implementing a monolithic, robot-specific solution, the system decomposes functionality into four interoperable and reusable frameworks: Smart Navigation, IoT Firmware Generation, RoboEmotion, and Supervisory Middleware. Each framework encapsulates a well-defined responsibility and exposes structured interfaces, enabling reuse across heterogeneous robotic platforms.

A key design contribution of ALTRUS is automatic code and package generation. Developers are not required to manually write ROS 2 packages, launch files, or navigation configurations. Instead, robot characteristics - including kinematic properties, sensor layouts, actuator limits, and hardware specifications - are defined using structured configuration files. From these parameters, the framework automatically generates consistent software artifacts required for execution. This approach eliminates fragmented coding practices and significantly reduces development effort. The generated artifacts can be executed in simulation environments for validation or deployed directly onto physical robots without modification, ensuring a seamless transition from virtual testing to real-world operation.

The ALTRUS robot serves as the reference implementation of the framework. All architectural components were implemented, integrated, and experimentally validated using this robot to demonstrate framework reusability, developer accessibility, and real-world feasibility.

Fig. 1 illustrates the high-level ALTRUS system architecture, including framework modularization, automated generation layers, and the perception–decision–execution data flow.

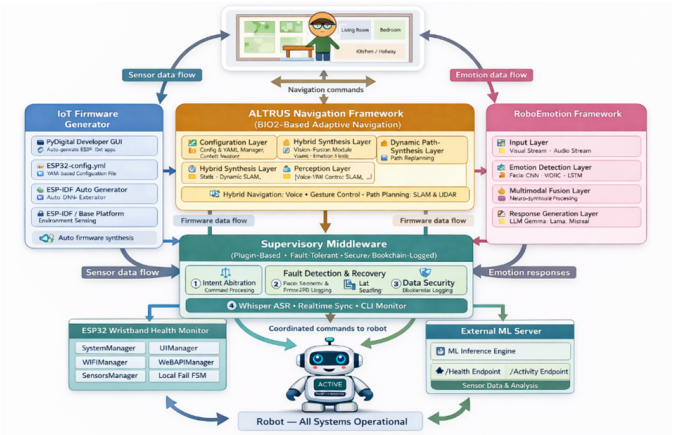

Fig. 1. High-level architecture of the integrated ALTRUS assistive robot system, illustrating the four interoperable software frameworks and their data flow relationships.

B. Core Components

1) Smart Navigation Framework

The Smart Navigation framework provides reusable and adaptive indoor mobility through a configuration-driven design, in which developers specify robot parameters declaratively. The framework automatically generates navigation-related software packages, configuration files, and launch artifacts.

Hybrid navigation integrates SLAM-based global localization with dynamic local replanning, enabling robust operation in changing indoor environments. Multimodal user intents derived from voice, gesture, or touchscreen inputs are normalized into structured commands and executed via deterministic state machines, ensuring predictable and safe robot motion.

By abstracting navigation logic from robot-specific implementations, the framework can be reused across different robotic platforms by simply modifying configuration parameters. Key capabilities include dynamic obstacle avoidance, which enables real-time adaptation to moving obstacles using local planners; intent prioritization, which resolves concurrent user commands based on assigned priority to prevent conflicts; and simulation compatibility, ensuring that generated ROS 2 packages can be executed in Gazebo or Webots for pre-deployment validation.

2) IoT Firmware Generation Framework

The IoT Firmware Generation framework provides automated and reusable embedded firmware development for ESP32-based devices. Hardware configurations, sensor mappings, communication protocols, and sampling rates are declared in structured configuration files.

Prior to code generation, configurations are validated for missing sensor definitions, invalid parameters, and registry or component mismatches. A template-based code generation engine produces complete ESP-IDF firmware projects while cleanly separating auto-generated artifacts from developer-authored logic, thereby preserving safe regeneration across iterations. The framework is extensible through a community package registry, in which new sensors are delivered as self-contained ESP-IDF components and installed without modifying the core generator.

This framework enables deterministic firmware reproduction and supports rapid adaptation to new sensor configurations, reducing manual implementation errors and overall development time. Notable features include automatic generation of FreeRTOS task scaffolding and inter-task communication hooks, registry-aware validation of sensor packages and dependency mappings, and modular component injection supporting heterogeneous ESP32 sensor configurations.

3) RoboEmotion Framework

The RoboEmotion framework enables reusable emotion-aware interaction through multimodal perception. Independent visual and auditory emotion recognition pipelines analyze facial expressions, voice tonality, and prosody, which are subsequently fused using confidence-aware strategies.

Emotion inference is represented in structured formats, allowing integration into robotic behaviors without requiring developers to implement low-level perception pipelines from scratch. A contextual memory module maintains emotional continuity across interactions, improving conversational coherence and human–robot rapport.

Additional features include multi-turn conversational context retention, probabilistic emotion fusion to handle uncertain sensory input, and an application programming interface (API) for triggering emotion-based navigation or interaction behaviors.

4) Supervisory Middleware

The Supervisory Middleware functions as a reusable coordination layer that ensures deterministic system behavior. Commands are represented as structured intents containing priority and severity metadata. Concurrent commands are arbitrated deterministically to guarantee predictable execution.

Subsystem health is monitored via heartbeat mechanisms and execution-time constraints. Fault detection triggers controlled recovery procedures, while a lightweight cryptographic hash-chain logging mechanism ensures tamper-evident execution traceability. Key contributions include deterministic arbitration of concurrent intents, real-time fault detection and recovery, and minimal-overhead runtime logging for regulatory compliance.

C. Experimental Methodology

The ALTRUS framework was implemented and evaluated using the ALTRUS robot as the reference platform. Experiments assessed developer usability, reproducibility, and deployment flexibility across four experimental categories.

Navigation experiments evaluated localization stability, obstacle avoidance performance, and intent response latency in both simulation and real indoor environments. Metrics included position root mean square error (RMSE), collision rate, and command execution latency. Firmware generation validation conducted repeatability testing and safe regeneration analysis on ESP32-based hardware, with verification covering functional correctness of drivers and timing compliance of FreeRTOS tasks. Emotion recognition evaluation tested performance under varying sensory conditions, including varying lighting, background noise, and facial occlusion, using metrics of recognition accuracy, response latency, and interaction coherence. Supervisory middleware testing employed controlled fault-injection experiments to measure arbitration latency, system recovery time, and logging integrity.

Results demonstrated high reliability, reproducibility, and seamless transition from simulation to real-world deployment across all evaluated components.

D. Implementation Details

The system was implemented using ROS 2 Foxy as the middleware backbone, leveraging its node-based communication model and real-time execution support. Navigation employs SLAM-based localization via GMapping with LiDAR input and A* path planning, integrated within the ROS 2 navigation stack. Embedded firmware is generated using ESP-IDF (v5.0) and FreeRTOS, with modular sensor components installable via the community registry and integrated through a standardized component interface. Emotion perception relies on machine learning models implemented in PyTorch for visual emotion recognition and the librosa library for auditory signal processing. The Supervisory Middleware is implemented as a ROS 2 node incorporating deterministic arbitration, heartbeat monitoring, and lightweight hash-chain logging.

The ALTRUS robot prototype consists of a differential-drive mobile base, a 2D LiDAR sensor for SLAM-based navigation, an onboard NVIDIA Jetson computing unit for perception and decision-processing modules, and ESP32-based wearable sensing modules for embedded IoT functionalities. All components were integrated and validated, collectively demonstrating the framework's real-world applicability, modularity, and developer accessibility.

IV. RESULTS AND DISCUSSION

The ALTRUS system integrates four frameworks: Smart Navigation, IoT Firmware Generation, RoboEmotion, and Supervisory Middleware. Results from individual and integrated evaluations are presented below.

A. Smart Navigation Framework

The framework was evaluated in a Gazebo simulation (8 m × 8 m) and on the physical ALTRUS prototype. Simulation trials (n = 52) achieved a 94.2% goal-reach success rate, 0.38 s mean planning time, 98.1% obstacle avoidance, and ±0.15 m AMCL localization error. Real-world trials (n = 40) recorded

a slight reduction to 87.5% success and ±0.24 m localization error, primarily due to RPLidar measurement uncertainty (±0.05 m) and odometry drift. Results are summarized in Tables I and II. Two collisions with low-profile obstacles (¡ 0.12 m) indicate the inflation radius should be increased to 0.15 m. The modular ROS 2 architecture enabled seamless simulation-to-real transition with minimal parameter changes, confirming framework reusability.

TABLE I
SIMULATION NAVIGATION PERFORMANCE

Metric	Value	Notes
Success Rate	94.2%	49/52 trials
Planning Time	0.38 s	NavFN planner
Navigation Time	12.4 s	Distance: 3–5 m
Obstacle Avoidance	98.1%	Inflation: 0.12 m
Localization Error	±0.15 m	AMCL SLAM
SLAM Coverage	96.7%	8 m × 8 m map

TABLE II
SIMULATION VS. REAL-WORLD NAVIGATION PERFORMANCE

Metric	Simulation	Real-World	Δ
Success Rate	94.2%	87.5%	-6.7 pp
Navigation Time	12.4 s	14.8 s	+19.4%
Localization Error	±0.15 m	±0.24 m	+0.09 m
Obstacle Avoidance	98.1%	91.2%	-6.9 pp

B. RoboEmotion Framework

1) Facial Emotion Recognition A CNN trained on FER2013 Plus (six classes) achieved 83–84% training and 76–77% validation accuracy, with strong performance on neutral and happy categories and expected confusion between sad/neutral and fear/surprise pairs.

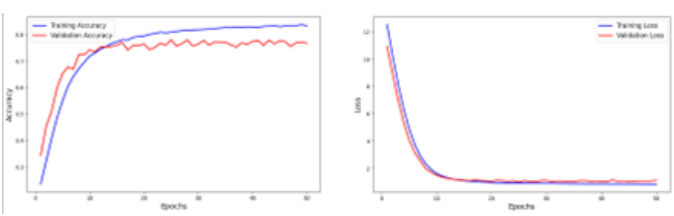

Fig. 2. Facial emotion recognition model training and validation loss and accuracy curves.

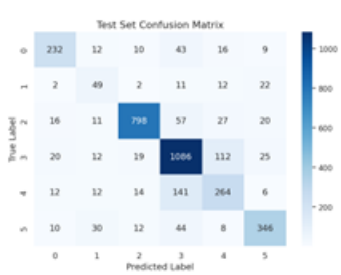

Fig. 3. Facial emotion recognition confusion matrix.

2) Speech Emotion Recognition Binary classification (happy vs. sad) reached approximately 90% validation accuracy after multi-class training failed on Sri Lankan voice samples due to dataset bias. Future work should incorporate region-specific datasets and multimodal fusion.

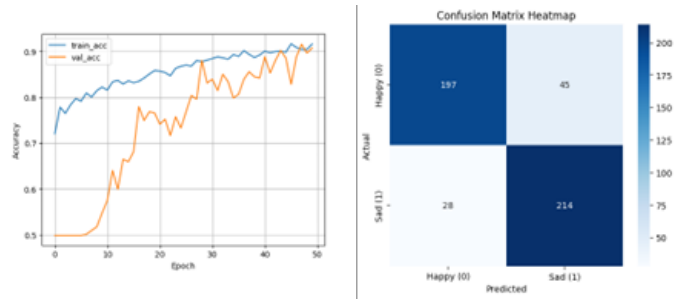

Fig. 4. Facial emotion recognition model training and validation loss and accuracy curves.

C. IoT Firmware Generation Framework

The framework (five sensor definitions, two display definitions, 39 Jinja2 templates) achieved a mean generation time of 54.09 ms. All community templates (2/2) passed validation; an invalid configuration was rejected in 0.09 ms. The GitHub-to-PlatformIO workflow reduces development friction, with marginal firmware size overhead in constrained profiles.

D. Health Risk Inference

Evaluated on 1,820 wristband samples (February 16 – March 6, 2026), classification yielded NORMAL (58.8%), WARNING (9.0%), and CRITICAL (32.3%), with a median decision latency of 1,091 ms (Table III). Context-aware logic correctly handled all high heart-rate running events as NORMAL, escalated all high heart-rate resting events, and suppressed low-SpO_2 motion artifacts. Only 2 cardiac-risk alerts were emitted from 198 eligible rows, confirming clinically conservative behavior.

TABLE III
CATEGORY COUNTS AND PROPORTIONS

Category	Count	Proportion
NORMAL	1,070	58.8%
WARNING	163	9.0%
CRITICAL	587	32.3%
Resting	1,400	76.9%
Running	325	17.9%
Unknown	95	5.2%

E. Supervisory Middleware

Fault-injection experiments confirmed deterministic intent arbitration, successful heartbeat-based fault detection with controlled recovery, and tamper-evident hash-chain logging with no integrity violations.

F. Integrated System Performance

All four frameworks operated concurrently without interference. The declarative configuration model reproduced settings consistently across simulation and real-world environments without source code modification, validating ALTRUS as a scalable and developer-accessible assistive robotic platform.

V. CONCLUSION

This paper presents ALTRUS, a modular, developer-oriented assistive robotic platform integrating four complementary software frameworks — Smart Navigation, IoT Health Sensor Integration, RoboEmotion, and Fault-Tolerant Middleware — into a unified and reusable architecture. Experimental evaluation on the ALTRUS robot prototype confirms that all four frameworks function correctly both independently and in full integration, demonstrating practical correctness, real-world usability, and developer-friendly adaptability in dynamic domestic environments.

The Smart Navigation Framework provides autonomous SLAM-based indoor navigation with LiDAR obstacle detection, real-time path replanning, and multimodal intent recognition through voice and gesture inputs. A declarative configuration pipeline automatically generates ROS 2 packages for both simulation and physical deployment. The IoT Health Sensor Integration Framework automates firmware generation for ESP32-based devices, supporting modular health and environmental sensing with configurable sampling and communication protocols, reducing repetitive implementation effort while preserving functional integrity. The RoboEmotion Framework fuses facial and vocal signals to track user emotional states and generate context-aware, empathetic responses via locally deployed large language models, while remaining modular and extensible for alternative models or interaction strategies. The Fault-Tolerant Middleware Framework ensures deterministic arbitration of concurrent commands, continuous subsystem health monitoring, emergency preemption, and blockchain-backed tamper-evident runtime logging, maintaining reliable operation under induced fault conditions.

Collectively, these frameworks establish a cohesive, adaptive, and resilient foundation that emphasizes reusability, modularity, and developer accessibility, supporting rapid customization across diverse robotic configurations and operational scenarios. Future work will focus on extending navigation to three-dimensional SLAM and predictive motion modeling, supporting additional microcontroller targets and secure transport protocols, expanding the community sensor plugin registry, improving large language model inference latency, and conducting large-scale real-world interaction studies to further validate framework scalability and effectiveness.

REFERENCES

[1] S. P. Thale, M. M. Prabhu, P. V. Thakur, and P. Kadam, "ROS based SLAM implementation for autonomous navigation using Turtlebot," *ITM Web Conf.*, vol. 32, art. 01011, 2020, doi:10.1051/itmconf/20203201011.

[2] C. Zhang, T. Huang, R. Zhang, and X. Yi, "PLD-SLAM: A new RGB-D SLAM method with point and line features for indoor dynamic scene," *ISPRS Int. J. Geo-Inf.*, vol. 10, art. 163, 2021, doi:10.3390/ijgi10030163.

[3] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020, arXiv:2003.00368.

[4] S. R. Sitaraman and H. M. Khalid, "Robotics automation and adaptive motion planning: A hybrid approach using AutoNav, LIDAR-based SLAM, and DenseNet with Leaky ReLU," *J. Trends Comput. Sci. Smart Technol.*, vol. 6, no. 4, pp. 404–423, Dec. 2024, doi:10.36548/jtcsst.2024.4.006.

[5] A. M. Fuad, S. J. Ahmed, N. J. Anannya, M. F. Mridha, and K. Nur, "An open-source voice command-based human-computer interaction system using speech recognition platforms," in *Proc. 2nd Int. Conf. Big Data, IoT and Machine Learning*, Lecture Notes in Networks and Systems, vol. 867, 2024, pp. 527–545, doi:10.1007/978-981-99-8937-9_36.

[6] J.-S. Han, C.-I. Lee, Y.-H. Youn, and S.-J. Kim, "A study on real-time hand gesture recognition technology by machine learning-based MediaPipe," *J. Syst. Manag. Sci.*, vol. 12, no. 2, pp. 462–476, Apr. 2022, doi:10.33168/JSMS.2022.0225.

[7] D. Ferrari, A. Pupa, A. Signoretti, and C. Secchi, "Safe multimodal communication in human-robot collaboration," in *Proc. Int. Workshop Human-Friendly Robotics (HFR)*, 2023, pp. 151–163, doi:10.1007/978-3-031-55000-3_11.

[8] P. Vanc and K. Stepanova, "TransforMerger: Transformer-based voice-gesture fusion for robust human-robot communication," arXiv:2504.01708, 2025.

[9] N. Harrand *et al.*, "ThingML: A language and code generation framework for heterogeneous targets," in *Proc. ACM/IEEE 19th Int. Conf. Model Driven Eng. Lang. Sys. (MODELS)*, 2016, pp. 597–607, doi:10.1145/2976767.2976812.

[10] M. Sharaf *et al.*, "Modeling and code generation framework for IoT (CAPSml)," 2019.

[11] Z. M. Korani, A. Moin, A. R. da Silva, and J. C. Ferreira, "Model-driven engineering techniques and tools for machine learning-enabled IoT applications: A scoping review," *Sensors*, vol. 23, no. 3, art. 1458, 2023, doi:10.3390/s23031458.

[12] J. Reyes-Ortiz *et al.*, "Human Activity Recognition Using Smartphones," UCI Machine Learning Repository, 2013.

[13] A. Sucerquia *et al.*, "SisFall: A fall and movement dataset," *Sensors*, vol. 17, no. 1, art. 198, 2017, doi:10.3390/s17010198.

[14] Y. Li *et al.*, "Lightweight CNN for real-time facial expression recognition," *Sensors*, 2022.

[15] B. Jiang, Y. Li, and W. Shen, "Real-time facial expression recognition for human–robot interaction using residual networks," *Appl. Sci.*, 2023.

[16] P. Tzirakis, J. Zhang, and B. W. Schuller, "End-to-end speech emotion recognition using deep neural networks," *IEEE J. Sel. Top. Signal Process.*, 2021.

[17] H. Li and Z. X. Li, "Context-aware speech emotion recognition," *Speech Commun.*, 2024.

[18] S. Anwar *et al.*, "Multimodal emotion recognition using facial and speech cues," *IEEE Access*, vol. 11, 2023.

[19] L. Chen *et al.*, "Transformer-based multimodal fusion for emotion recognition," *Sensors*, 2024.

[20] T. B. Brown, "Language models as a tool for conversational AI," arXiv preprint, 2023.

[21] M. Quigley *et al.*, "ROS: an open-source Robot Operating System," in *Proc. IEEE Int. Conf. Robotics and Automation Workshop on Open Source Software*, 2009.

[22] Y. Maruyama *et al.*, "Exploring the performance of ROS2," in *Proc. 13th Int. Conf. Embedded Software (EMSOFT)*, 2016, pp. 1–10, doi:10.1145/2968474.2968560.

[23] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf.

[24] K. Christidis and M. Devetsikiotis, "Blockchains and smart contracts for the Internet of Things," *IEEE Access*, vol. 4, pp. 2292–2303, 2016, doi:10.1109/ACCESS.2016.2566339.